CS 8803 GPU Project 2 – Bitonic Sort using GPU

Saikrishnan Sankar
College of Computing
Georgia Institute of Technology
Atlanta, USA
ssankar49@gatech.edu

Abstract

The report explains the implementation of Bitonic Sort on the H100 GPU and discusses the optimizations made using CUDA to improve performance. This algorithm achieves 1049 Million Elements Per Second (MEPS) GPU sort speed, 20ms Host-to-Device transfer time, 66ms kernel time, and 7ms Device-to-Host transfer time for an input size of 100M elements.

The Bitonic Sort Algorithm

Bitonic sort works by first creating bitonic sequences from the input array. A bitonic sequence increases then decreases (or vice versa). The algorithm repeatedly splits these bitonic sequences into smaller subsequences using compare-and-swap operations, where elements are compared and swapped based on their positions. Through successive splitting and merging operations, the entire array gets sorted while maintaining the parallel nature that makes it perfect for GPU implementation.

Algorithm 1 Bitonic Sort Algorithm CPU - Given to us

```
for i = 1 to log n do
  for j = i − 1 to 0 do
    for k = 0 to n − 1 do
      a ← arr[k]
      b ← arr[k ⊕ 2^j]
      if (k ⊕ 2^j) > k then
        {(a, b) are compared so skip (b, a) case}
        if (2^i & k) = 0 then
          Compare_Exchange_↑(a, b)
        else
          Compare_Exchange_↓(a, b)
        end if
      end if
    end for
  end for
end for
```

Kernel Optimizations

The given CPU algorithm can be parallelized. Here, we can remove the innermost loop and run it using n workers on the GPU. By simply padding the input (to account for all sizes), we observe 120 MEPS only for a 100M input size. There are several bottlenecks here:

1. Number of kernel calls = O($\log^2$ n). This can be optimized with the use of shared memory. The maximum number of threads a block can hold is 1024 for the H100 GPU, so BLOCK_SIZE is set to 1024. Therefore, whenever idx and partner (elements to compare) are within the same block, we can compare through shared memory.

2. The above optimization reduces the number of kernel calls, but there are still many global memory accesses across blocks. What if each thread can access and work on multiple elements? The H100 GPU has 48 KB of shared memory per streaming multiprocessor (SM). So we can store up to 2^{13} elements (32 KB) – 8192 elements in shared memory per block ($2^{14} = 64$ KB). Therefore, each thread will work on 4 strides (Total memory / Block size). Initially, we perform bitonic sort on N blocks (total input size / 8192). Now one kernel call will sort each of these blocks in ascending and descending order sequences. For across-block comparisons, we do have to use global memory until $J >$ (shared memory size). Then, for the remaining comparisons, we go back to shared memory, which is much faster.

   My GPU bitonic sort algorithm works in two main phases:

   (a) Phase 1: Initial Shared Memory Sort

      i. Divide the input array into blocks of size 2^{13} (typically 8192 elements)

ii. Each GPU block handles one array block using shared memory

iii. Each thread within a block processes multiple elements (strides = 4)

iv. Perform complete bitonic sort within each block:

- Build bitonic sequences of increasing sizes (2, 4, 8, ..., block_size)
- For each sequence size, merge adjacent sequences
- Alternate between ascending and descending order to maintain bitonic property

v. All comparisons happen in fast shared memory since elements are within the same block

(b) Phase 2: Hybrid Global-Shared Memory Merge

i. For sequence sizes larger than 2^{13}:

- Use global memory for cross-block comparisons
- Launch kernels that compare elements across different GPU blocks
- Each thread compares elements that are far apart in the array

ii. When comparison distance becomes small enough (fits in shared memory):

- Switch back to shared memory operations
- Load relevant array sections into shared memory
- Perform final merge operations using fast shared memory

iii. Repeat this hybrid approach for each bitonic sequence size until the entire array is sorted

3. Due to the nature of the bitonic sort algorithm, many threads would normally remain idle. In a naive implementation, when finding comparison partners, for idx = 0 and j = 0, the partner is 1. Similarly, for idx = 1 and j = 0, the partner is 0. Since this pair was already compared by thread 0, thread 1 performs redundant work. A smart solution ensures all threads perform useful work while achieving memory coalescing.

The optimized index calculation uses:

$$idx_1 = \text{thread_id} + (\text{thread_id} \wedge \neg((1 \ll j) - 1)) \tag{1}$$

$$idx_2 = idx_1 + (1 \ll j) \tag{2}$$

This works because $(\text{thread_id} \wedge \neg((1 \ll j) - 1))$ masks out the lower j bits of the thread ID, effectively grouping threads into blocks of size 2^j. Within each block, threads are assigned unique comparison pairs. For example, when $j = 1$ and thread_id = 2: $idx_1 = 2 + (2 \wedge \neg 1) = 2 + 2 = 4$ and $idx_2 = 4 + 2 = 6$. This ensures every thread compares a distinct pair of elements, eliminating redundant work while maintaining coalesced memory access patterns since consecutive threads access nearby memory locations.

4. Loop unrolling is critical for GPU performance optimization. The `#pragma unroll` directive instructs the CUDA compiler to completely unroll loops at compile time, eliminating loop overhead such as counter increments, condition checks, and branch instructions. In bitonic sort, the number of iterations for inner loops is typically small and known at compile time (e.g., `13` iterations). By unrolling these loops, we reduce branch divergence within warps, allow better instruction-level parallelism, and enable the compiler to perform more aggressive optimizations like register allocation and instruction scheduling.

H2D & D2H Optimizations

The initial approach used basic `cudaMemcpy()` calls for H2D and D2H transfers. After discussions on Ed, several bottlenecks in memory transfer became apparent that were killing performance. The main issue? Regular `cudaMemcpy()` is incredibly slow because it goes through an extra step - host data gets copied to a temporary pinned buffer first, then DMA transfers it to GPU.

The obvious fix is pinned memory allocation using `cudaMallocHost()`, but this hit a roadblock since `arrCpu` and `arrSortedGpu` were already allocated with `malloc()` in code that couldn't be modified. The solution: `cudaHostRegister()` to retrospectively pin existing allocations. This eliminates the intermediate copy step and enables direct DMA transfers.

To squeeze out more performance, several tricks were implemented:

1. Pin `arrSortedGpu` during kernel execution to overlap computation with memory operations
2. Replace synchronous `cudaMemcpy()` with `cudaMemcpyAsync()` for non-blocking transfers
3. Remove `cudaDeviceSynchronize()` calls that were stalling the pipeline
4. Overlap H2D transfer of next batch while GPU processes current data

The performance difference is dramatic. Without pinned memory, every transfer involves an extra memcpy operation on the host side. With pinned memory and async transfers, most of the memory latency can be hidden behind computation, essentially getting "free" data movement while the GPU is busy sorting.

RESULTS AND INFERENCES

First, I have shown the baseline results, which represent a simple CUDA program performing bitonic sort without shared memory or any optimizations.

```
FUNCTIONAL SUCCESS
Array size          : 100000000
CPU Sort Time (ms)  : 17194.759766
GPU Sort Time (ms)  : 397.757294
GPU Sort Speed      : 251.409592 million elements per second
PERF PASSING
GPU Sort is  43x faster than CPU !!!
H2D Transfer Time (ms): 207.960922
Kernel Time (ms)      : 101.842560
D2H Transfer Time (ms): 87.953827
```

Figure 1: Baseline results

We can clearly see that the sort speed is quite low, and kernel time, D2H, and H2D transfer times are significantly high. The image below captures the results after applying all optimizations discussed in this report.

```
FUNCTIONAL SUCCESS
Array size          : 100000000
CPU Sort Time (ms)  : 18283.644531
GPU Sort Time (ms)  : 95.253342
GPU Sort Speed      : 1049.831909 million elements per second
PERF PASSING
GPU Sort is  191x faster than CPU !!!
H2D Transfer Time (ms): 20.922432
Kernel Time (ms)      : 66.790878
D2H Transfer Time (ms): 7.540032
```

Figure 2: Results for final

The final implementation shows very high performance, with a sort speed of 1049 MEPS. This improvement is mainly due to accessing shared memory for 8192 elements. Furthermore, we can see that the H2D and D2H times have decreased significantly due to memory pinning and asynchronous transfers.

Nsight Compute Results

The following section contains screenshots of performance counter analysis made using Nsight Compute for throughput and occupancy for the kernels used for bitonic sort. A total of 3 kernels were used: the first kernel (`bitonic_sort_shared_initial`) performs complete bitonic sort within shared memory blocks for sequences up to 2^{13} elements, handling all sorting and merging operations for small sequences entirely in fast shared memory. The second kernel (`bitonic_merge_global`) handles cross-block comparisons using global memory when sequence sizes exceed shared memory capacity, performing merge operations with large strides across different GPU blocks. The third kernel (`bitonic_sort_shared_final`) performs the final shared memory optimization step after global memory merges, bringing elements back into shared memory for efficient local comparisons when the stride becomes small enough to fit within a block's shared memory.

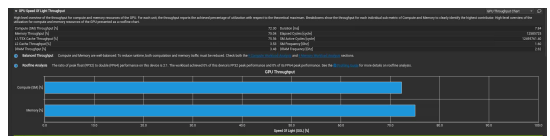

Figure 3: Throughput analysis for Shared initial kernel

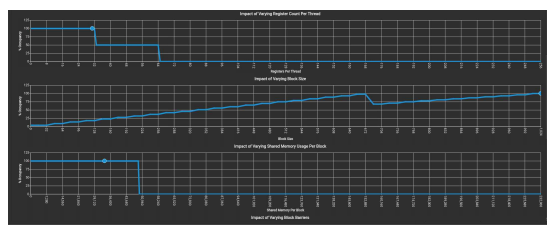

Figure 4: Occupancy analysis for Shared initial kernel

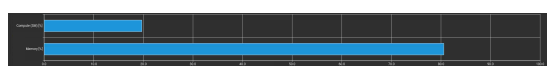

Figure 5: Throughput analysis for Global kernel

When we look at throughput, the global memory kernel (`bitonic_merge_global`) has lower compute usage at 72% compared to 75% memory throughput. This makes sense because it's constantly accessing global memory with messy stride patterns. The shared memory kernels (`bitonic_sort_shared_initial` and `bitonic_sort_shared_final`) work much better since they stay in fast shared memory the whole time.

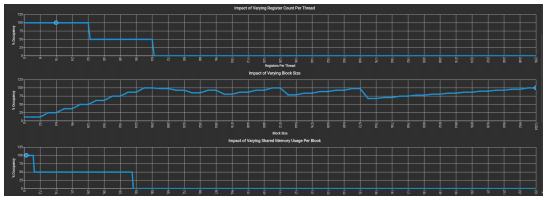

Figure 6: Occupancy analysis for Global kernel

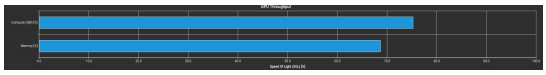

Figure 7: Throughput analysis for Shared Final Kernel

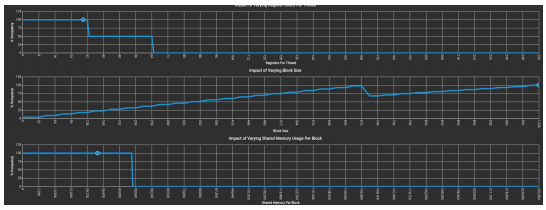

Figure 8: Occupancy analysis for Shared final kernel

The initial shared memory kernel does more work because it builds complete bitonic sequences from size 2 up to the full block size. The final one mostly just merges with smaller strides.

The register analysis shows a big problem - performance crashes hard after 24 registers per thread. This means register spilling kills performance. For block sizes, we get best results around 512-1024 threads per block. Using 1024 threads gives us maximum occupancy, which is why we picked that.

The biggest limit is shared memory - it hits a wall at 32KB per block. Go over that and occupancy drops to zero. That's why using 8192 elements (32KB) for shared memory is perfect for this GPU.

Future work & Inference

This project was fascinating as it exposed me to low-level programming using CUDA and various optimization techniques. Furthermore, I became familiar with NVIDIA Nsight Compute profiling, which is used extensively in the industry. I enjoyed working with threads and optimizing shared memory usage, as well as reducing branch divergence for GPU operations. My algorithm can be scaled by addressing the register constraints mentioned above and including warp-level optimizations (essentially hardware optimizations) to achieve even higher performance.

References

[1] NVIDIA Corporation, "CUDA C++ Programming Guide," NVIDIA Developer Documentation.

[2] NVIDIA Corporation, "NVIDIA Nsight Compute," NVIDIA Developer.